Clasificación de imágenes de anime, caricaturas y personas mediante redes neuronales convolucionales

Manuel Bajos Rivera, A01711581

***Abstract*— En este proyecto se realiza la clasificación automática de imágenes en tres categorías: anime, caricaturas (cartoon) y personas (human). Para ello se utiliza el conjunto de datos Anime vs Cartoon vs Human obtenido de Kaggle, compuesto por 8,917 imágenes distribuidas de forma casi balanceada entre las tres clases (2913 cartoon, 3004 anime y 3000 human).**

El proyecto se enfoca en un problema de clasificación multiclase de imágenes. Se implementa primero una red neuronal convolucional propia con TensorFlow y Keras como modelo base, y posteriormente se mejora mediante técnicas de regularización, ajuste de hiperparámetros y transfer learning con una arquitectura preentrenada. Un análisis exploratorio previo identificó 383 imágenes duplicadas exactas y diferencias marcadas de resolución y peso entre clases, que podrían permitir al modelo clasificar por atajos y no por contenido.

[PENDIENTE: resumir los resultados finales: exactitud en prueba de la CNN base, de la CNN mejorada y del modelo con transferencia de aprendizaje, y qué cambios funcionaron.]

I. Problemática

Distinguir automáticamente el estilo visual de una imagen (dibujo de anime, caricatura occidental o fotografía de una persona) es un problema de clasificación de imágenes que puede resultar útil, por ejemplo, para organizar colecciones de imágenes o filtrar contenido. Las tres categorías comparten elementos como rostros, colores y contornos, pero difieren en el trazo, la textura y la composición, por lo que no es una tarea trivial para un clasificador basado en reglas.

Dado esto, se plantea un problema de clasificación multiclase cuya meta es predecir, a partir de una imagen a color, a cuál de las tres clases pertenece: `anime`, `cartoon` o `human`. Las variables de entrada son los píxeles de la imagen y la variable objetivo es la etiqueta de la clase.

Dado que los datos son imágenes, se emplean redes neuronales convolucionales (CNN), que son capaces de aprender de forma automática jerarquías de características visuales a partir de los píxeles.

II. Datos

El conjunto de datos utilizado corresponde a Anime vs Cartoon vs Human, disponible en Kaggle [1]. Contiene 8,917 imágenes organizadas en tres carpetas, una por clase. La Tabla I resume su composición.

TABLE I

Composición del conjunto de datos original

Clase	Imágenes	Formato	Ancho típico	Alto típico	Peso típico
`anime`	3,004	JPEG	68 px	68 px	4 KB
`cartoon`	2,913	PNG y JPEG	258 px	180 px	55 KB
`human`	3,000	PNG	1,024 px	1,024 px	1,334 KB

La Figura 1 muestra que las clases están casi balanceadas, con entre 2,913 y 3,004 imágenes por clase, por lo que la exactitud (accuracy) es una métrica adecuada para comparar los modelos. Sin embargo, como la exactitud global puede ocultar un desempeño deficiente en una clase particular, se reportan además la precisión, la sensibilidad (recall) y la medida F1 por clase, así como el F1 macro, que promedia las tres clases con el mismo peso, y la matriz de confusión.

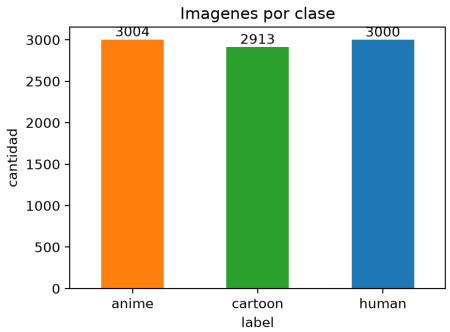


Fig. 1. Número de imágenes por clase en el conjunto original.

Las imágenes tienen características muy distintas entre clases. Las imágenes de anime son miniaturas cuadradas de entre 25 y 189 píxeles de ancho. Las de cartoon van de 74 a 519 píxeles de ancho y su relación de aspecto varía entre 0.37 y 5.35, de modo que solamente el 12 % son cuadradas. Las de human son siempre cuadradas de 1,024 × 1,024 píxeles.

Además, las imágenes de cartoon mezclan tres modos de color: 653 en RGB, 2,186 en RGBA y 74 con paleta indexada. Ninguna de las imágenes con canal alfa presenta transparencia real, por lo que su conversión a RGB no altera el contenido.

III. Análisis exploratorio

Antes del entrenamiento se realizó un análisis de la integridad, la resolución, el color y el contenido de las imágenes. Se verificó que ninguna de las 8,917 imágenes estuviera corrupta.

La Figura 2 presenta una muestra aleatoria de imágenes de cada clase, con su resolución original. Se observa que tanto anime como human son retratos de rostros recortados, mientras que cartoon incluye personajes de cuerpo completo, escenas y capturas de pantalla con márgenes. Esto indica que, además del estilo, la composición de la imagen también diferencia a las clases.

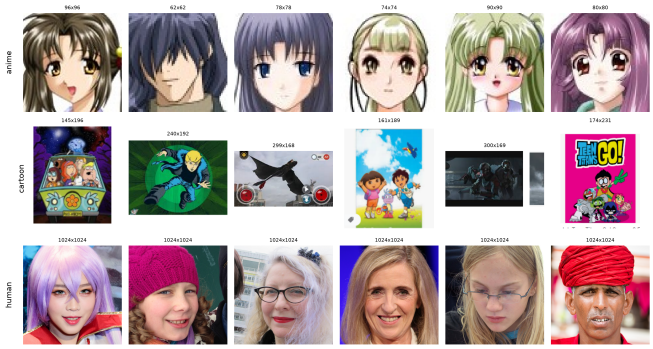


Fig. 2. Muestra aleatoria de imágenes por clase. El título de cada imagen indica su resolución original.

III-A. Riesgo de atajos por resolución y peso

La Figura 3 compara el valor típico (mediana) de cada clase. Las imágenes de human son aproximadamente 15 veces más anchas y 300 veces más pesadas que las de anime. Esta diferencia es tan marcada que, por sí sola, permitiría identificar la clase sin observar el contenido de la imagen.

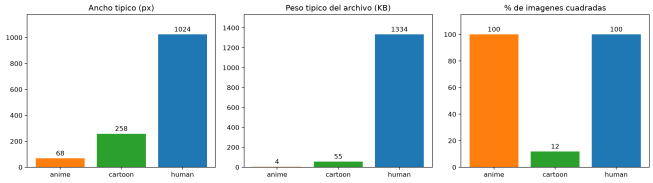


Fig. 3. Ancho típico, peso típico del archivo y porcentaje de imágenes cuadradas por clase.

Esto representa un riesgo, ya que un modelo podría aprender un atajo basado en la nitidez o la resolución y no en las características visuales de cada categoría, lo que produciría una exactitud alta pero engañosa. Para mitigarlo, todas las imágenes se redimensionan a un mismo tamaño antes de entrenar, y se evalúa el modelo con imágenes externas que no provienen del conjunto de datos.

Una diferencia similar aparece en el color. La Figura 4 muestra que las imágenes de human son en promedio más oscuras (medias de los canales R, G y B de 0.52, 0.42 y 0.38) que las de anime (0.64, 0.54 y 0.51) y cartoon (0.61, 0.58 y 0.55), lo que constituye otra señal que el modelo podría aprovechar.

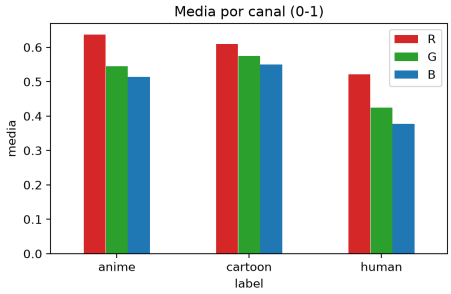


Fig. 4. Media de cada canal de color por clase, sobre una muestra de 300 imágenes por clase.

III-B. Imágenes duplicadas

Se calculó el resumen criptográfico MD5 del contenido de cada archivo para detectar duplicados exactos. Se encontraron 712 imágenes involucradas en 329 grupos de duplicados, es decir, 383 imágenes sobrantes. Todas pertenecen a la clase anime y ningún grupo mezcla clases distintas. La Figura 5 muestra algunos ejemplos: son la misma imagen guardada con nombres de archivo diferentes. La mayoría de los grupos son pares, aunque algunos contienen hasta cinco copias.

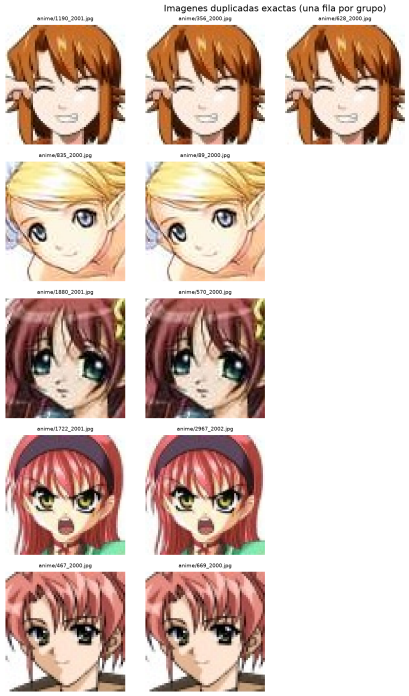


Fig. 5. Ejemplos de grupos de imágenes duplicadas exactas (una fila por grupo).

Los duplicados deben eliminarse antes de dividir los datos. De lo contrario, una copia podría quedar en el conjunto de entrenamiento y otra en el de prueba, y el modelo habría visto la respuesta durante el entrenamiento, con lo que el desempeño en prueba sería artificialmente alto. Tras la deduplicación el conjunto contiene 2,621 imágenes de anime, 2,913 de cartoon y 3,000 de human (8,534 en total), con un desbalance leve.

IV. ETL

El proceso de extracción, transformación y carga se realizó antes del entrenamiento con el objetivo de obtener un conjunto de datos adecuado para la clasificación. El proyecto se implementó en Python con TensorFlow 2.20 y Keras 3 [2], [3], versiones que cuentan con soporte en macOS, Linux y Windows.

La extracción se hizo mediante la biblioteca `kagglehub`, que descarga el conjunto de datos y lo guarda en una caché local. Posteriormente se construyó una tabla con la ruta y la clase de cada imagen, y se eliminaron los duplicados exactos como se describió en la sección anterior.

IV-A. Partición de los datos

Sobre las imágenes sin duplicados se realizó una partición estratificada por clase en tres subconjuntos: 70 % para entrenamiento, 15 % para validación y 15 % para prueba, con una semilla fija (42) para que los resultados sean reproducibles. La estratificación mantiene en cada subconjunto la misma proporción de clases que en el total, lo que conserva el leve desbalance de anime. La Tabla II muestra el resultado.

TABLE II
NÚMERO DE IMÁGENES POR CLASE Y PARTICIÓN

Clase	Entrenamiento	Validación	Prueba	Total
`anime`	1,834	393	394	2,621
`cartoon`	2,039	437	437	2,913
`human`	2,100	450	450	3,000
Total	5,973	1,280	1,281	8,534

La partición se hizo sobre las rutas de las imágenes y antes de transformarlas, de modo que ninguna información de validación o prueba interviene en el entrenamiento. Además, se verificó mediante el resumen MD5 que ninguna imagen aparece en más de una partición. El conjunto de entrenamiento se utiliza para ajustar los parámetros de los modelos, el de validación para monitorear el desempeño y ajustar hiperparámetros, y el de prueba se mantiene aislado y se usa únicamente para reportar el desempeño final.

IV-B. Transformación de las imágenes

Cada imagen se convierte a RGB de tres canales y se deja en un tamaño único de 128×128 píxeles, ya que la red convolucional requiere entradas de dimensiones fijas. El redimensionado se realiza con el filtro de Lanczos de la biblioteca Pillow [4], un método de remuestreo por convolución con antialiasing, aplicado de la misma manera a todas las clases. Esto es relevante porque reducir imágenes sin un filtro adecuado puede introducir aliasing [5], y en este conjunto de datos la reducción es considerable, por ejemplo de 1,024 a 128 píxeles en las imágenes de human.

Como las imágenes de cartoon no son cuadradas en su mayoría, para llegar a un cuadrado se utiliza relleno (padding): se reduce la imagen conservando su proporción, de modo que su lado mayor mida 128 píxeles, y se centra sobre un lienzo cuadrado rellenado con el color mediano de su propio borde, para que las bandas resultantes sean poco llamativas. De esta forma no se deforman las imágenes.

Sin embargo, como solo el 12 % de las imágenes de cartoon son cuadradas y el 100 % de anime y human lo son, el relleno hace que casi todas las imágenes de cartoon tengan bandas y ninguna de las otras clases, lo cual puede convertirse en un atajo para el modelo. Por ello, en una sección posterior se compara con la alternativa de estirar la imagen a un cuadrado.

IV-C. Carga y almacenamiento en caché

Las imágenes transformadas se guardan como arreglos de enteros sin signo de 8 bits (`uint8`) en una caché local, uno por partición. De esta forma, el costo de decodificar y reducir las imágenes originales, en particular las de 1,024 $\times$ 1,024 píxeles de human, se paga una sola vez. El conjunto de entrenamiento ocupa aproximadamente 294 MB en memoria, por lo que cabe completo en la RAM. El procesamiento de las 8,534 imágenes se paraleliza con hilos y tomó 11 segundos en la computadora utilizada. Las etiquetas se codifican como enteros (0 para anime, 1 para cartoon y 2 para human).

La normalización de los píxeles al rango $[0, 1]$ se realiza dividiendo entre 255 mediante una capa `Rescaling` dentro de la CNN propia. Para la arquitectura preentrenada (EfficientNetB0) no se divide entre 255: la versión de Keras incluye en sus primeras capas el reescalado y la normalización de la entrada, y espera píxeles en el rango $[0, 255]$, que es como se guardan las imágenes en la caché.

V. MODELOS

V-A. CNN propia (modelo base)

Como primer modelo se diseñó una red neuronal convolucional sencilla y se entrenó desde cero, sin regularización ni aumento de datos, para tener un punto de comparación claro. Cada mejora posterior se prueba por separado, de modo que se pueda explicar qué cambio funcionó y por qué.

La red recibe imágenes RGB de 128×128 píxeles y está formada por tres bloques convolucionales, cada uno con una capa convolucional y una de agrupación máxima, seguidos de un clasificador denso. La Tabla III muestra sus capas.

TABLE III
ARQUITECTURA DE LA CNN BASE

Capa	Salida	Parámetros
Reescalado ($\div 255$)	$128 \times 128 \times 3$	0
Conv2D, 32 filtros 3×3, ReLU	$126 \times 126 \times 32$	896
MaxPooling2D 2×2	$63 \times 63 \times 32$	0
Conv2D, 64 filtros 3×3, ReLU	$61 \times 61 \times 64$	18,496
MaxPooling2D 2×2	$30 \times 30 \times 64$	0
Conv2D, 128 filtros 3×3, ReLU	$28 \times 28 \times 128$	73,856
MaxPooling2D 2×2	$14 \times 14 \times 128$	0
Flatten	25,088	0
Dense, 128 neuronas, ReLU	128	3,211,392
Dense, 3 neuronas, softmax	3	387
Total		**3,305,027**

La capa convolucional aplica a la imagen un conjunto de filtros pequeños cuyos valores se aprenden. Para un filtro K de tamaño $3 \times 3 \times C$ y un sesgo b, la salida en la posición (i, j) es

$$z_{i,j} = b + \sum_{c=1}^{C} \sum_{m=0}^{2} \sum_{n=0}^{2} K_{m,n,c}\, x_{i+m,\, j+n,\, c}, \qquad (1)$$

y sobre ella se aplica la función de activación ReLU, $\mathrm{ReLU}(z) = \max(0, z)$. Las primeras capas detectan rasgos simples, como bordes y texturas, y las siguientes los combinan en formas más complejas. La agrupación máxima de 2×2 conserva el mayor valor de cada ventana y reduce a la mitad el tamaño espacial, lo que disminuye el número de parámetros y da cierta tolerancia a pequeños desplazamientos.

La última capa tiene tres neuronas con activación softmax, que convierte los valores z_1, z_2, z_3 en probabilidades de cada clase:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{3} e^{z_j}}. \tag{2}$$

Como las etiquetas son números enteros, se usa la entropía cruzada categórica dispersa, que penaliza la probabilidad baja que el modelo asigna a la clase correcta y_i:

$$L = -\frac{1}{N} \sum_{i=1}^{N} \log p_{i,y_i}. \tag{3}$$

El entrenamiento utiliza el optimizador Adam con una tasa de aprendizaje de 0.001, lotes de 32 imágenes y 15 épocas.

V-B. *Evaluación del modelo base*

La Figura 6 muestra la evolución del entrenamiento. La exactitud en entrenamiento llega a 100 % desde la época 8, y la pérdida de validación baja hasta 0.004 y se mantiene prácticamente estable (entre 0.0040 y 0.0044), por lo que no se observa el deterioro de la validación típico del sobreajuste.

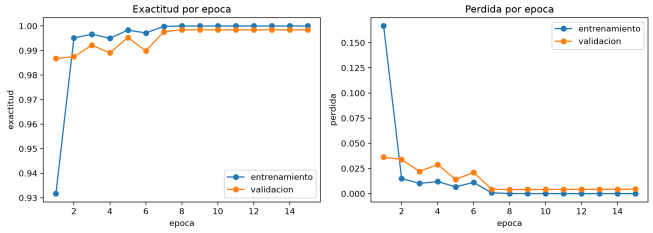


Fig. 6. Exactitud y pérdida por época de la CNN base, en entrenamiento y validación.

En el conjunto de validación el modelo obtuvo una exactitud de 99.84 % (F1 macro de 0.9985) y en el de prueba de 99.53 % (F1 macro de 0.9953), con un F1 de 0.996, 0.993 y 0.997 para anime, cartoon y human, respectivamente. La matriz de confusión en prueba (Figura 13) muestra 6 errores en 1,281 imágenes: 3 de anime clasificadas como cartoon, 2 de cartoon clasificadas como human y 1 de human clasificada como cartoon.

Estos valores son muy altos, pero, como se muestra en la sección de predicciones con imágenes nuevas, solo describen el desempeño con imágenes parecidas a las del conjunto de datos.

V-C. *Transferencia de aprendizaje*

Como la CNN base solo acertó 2 de las 8 imágenes externas, se probó una red preentrenada. La transferencia de aprendizaje aprovecha las características visuales que aprendió una red entrenada con un conjunto de datos muy grande: las primeras capas detectan rasgos generales, como bordes y texturas, que sirven para muchas tareas, y solo las últimas capas se ajustan al problema nuevo. Gando et al. [6] aplican esta estrategia a un problema cercano, distinguir ilustraciones de fotografías: ajustan las últimas capas de AlexNet preentrenada con ImageNet, con 20,000 imágenes, y reportan una exactitud de 96.8 %, superior a la de las redes convolucionales entrenadas desde cero que compararon, la mejor de las cuales alcanzó 93.0 %.

Se utilizó EfficientNetB0 [7] con los pesos preentrenados en ImageNet [8]. EfficientNet escala de forma conjunta la profundidad, el ancho y la resolución de la red, y la versión B0 tiene 4,049,571 parámetros sin su clasificador original. Se le quitó ese clasificador y se añadió una cabeza nueva, formada por una capa de agrupación promedio global y una capa densa de 3 neuronas con softmax (3,843 parámetros). Como se explicó en la sección de ETL, esta red recibe los píxeles en el rango $[0, 255]$ sin dividirlos entre 255.

El entrenamiento se hizo en dos fases. En la primera, la base se mantuvo congelada y solo se entrenó la cabeza, durante 8 épocas con Adam y una tasa de aprendizaje de 0.001. En la segunda se descongelaron las últimas 20 de las 238 capas de la base, dejando congeladas las capas de normalización por lotes, y se continuó el entrenamiento durante 10 épocas con una tasa de aprendizaje mucho menor (0.00001), para no alterar bruscamente los pesos preentrenados; en esta fase se entrenan 1,346,611 parámetros. Para poder atribuir los cambios a la red preentrenada, se usaron las mismas imágenes y particiones que en la CNN base, sin aumento de datos ni regularización adicional.

La Figura 7 muestra el entrenamiento. En validación el modelo alcanzó una exactitud de 99.92 % (F1 macro de 0.9992) y en prueba de 99.92 % (F1 macro de 0.9992), con un único error en prueba, una imagen de human clasificada como cartoon (Figura 13), frente a los 6 errores de la CNN base. Los resultados con imágenes externas se presentan en la sección de predicciones con imágenes nuevas.

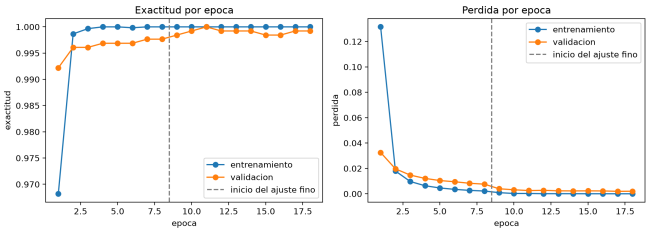


Fig. 7. Exactitud y pérdida por época de EfficientNetB0. La línea punteada marca el inicio del ajuste fino.

V-D. *Regularización con Dropout*

Ninguno de los dos modelos anteriores incluye regularización explícita. Dropout [9] apaga al azar una fracción de las neuronas de una capa en cada paso de entrenamiento: cada neurona se descarta con probabilidad r y las restantes se escalan por $1/(1-r)$. De esta forma la red no puede depender de neuronas individuales y se reduce la coadaptación entre ellas. Solo actúa durante el entrenamiento; al predecir se usan todas las neuronas.

Se probó por separado en cada arquitectura, con una sola tasa por modelo y sin ajustarla, para no sobreajustar a un conjunto externo tan pequeño. En la CNN se añadió con $r = 0.5$ después de la capa densa de 128 neuronas, que

concentra el 97 % de los parámetros de la red; 0.5 es el valor habitual para capas densas. En EfficientNetB0 se añadió con $r = 0.2$ después de la agrupación promedio global, la misma tasa que usa el clasificador original de esa red. El resto del entrenamiento (datos, particiones, épocas, tasas de aprendizaje y fases) fue idéntico al de los modelos sin Dropout, de modo que la única diferencia es esa capa. Cada modelo se entrenó una sola vez.

La Figura 8 muestra el entrenamiento. En la CNN, el Dropout tuvo el efecto esperado sobre las curvas: la pérdida de entrenamiento al final de las 15 épocas es de 5.5×10^{-4}, frente a 6.3×10^{-6} sin Dropout, y las curvas son más irregulares, porque cada paso usa una red distinta. Sin embargo, esto no se tradujo en mejores métricas de validación: la exactitud fue la misma (99.84 %) y la pérdida final de validación fue algo mayor (0.0067 frente a 0.0044). En EfficientNetB0 los dos entrenamientos fueron prácticamente iguales (pérdida final de validación de 0.0025 con Dropout y 0.0020 sin él).

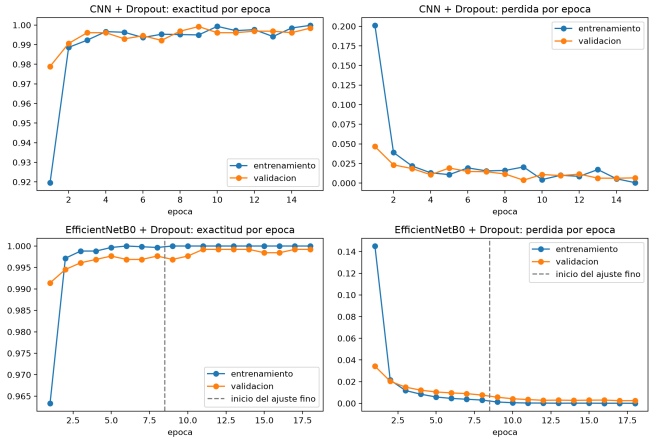


Fig. 8. Exactitud y pérdida por época de los modelos con Dropout. En EfficientNetB0, la línea punteada marca el inicio del ajuste fino.

En el conjunto de prueba, la CNN con Dropout cometió 5 errores (exactitud de 99.61 %) frente a 6 sin Dropout, y EfficientNetB0 con Dropout no cometió ninguno (100 %) frente a 1 sin él (Figura 9). Son diferencias de una sola imagen en 1,281, que no permiten afirmar una mejora, y como cada modelo se entrenó una vez, también pueden deberse a la aleatoriedad del entrenamiento.

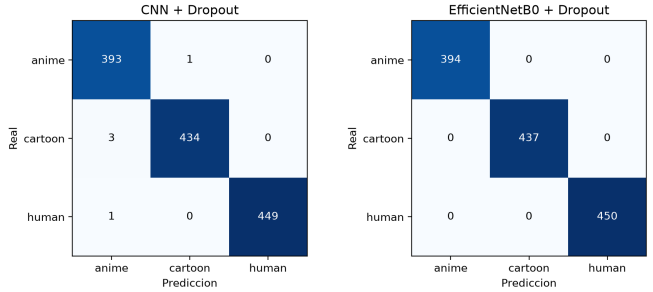


Fig. 9. Matrices de confusión de los modelos con Dropout en el conjunto de prueba.

Con las imágenes externas, ningún modelo cambió su clase predicha en ninguna de las 8 imágenes (Tabla IV): la CNN con Dropout sigue acertando 2 de 8 (25.0 %) y EfficientNetB0 con Dropout 4 de 8 (50.0 %), con los mismos aciertos y errores que sin Dropout. La confianza cambió poco (por ejemplo, la viñeta de manga, clasificada erróneamente como cartoon por EfficientNetB0, pasó de 94 % a 98 % de confianza) y los errores de la CNN siguen teniendo una confianza cercana a 100 %.

TABLE IV
PREDICCIONES DE LOS MODELOS CON DROPOUT SOBRE IMÁGENES EXTERNAS (ENTRE PARÉNTESIS, LA CONFIANZA)

Imagen	Real	CNN + Dropout	EfficientNetB0 + Dropout
`Caricaturas2.jpg`	cartoon	cartoon (100 %)	cartoon (100 %)
`caricatura1.jpg`	cartoon	cartoon (100 %)	cartoon (100 %)
`pruebaAnime.jpg`	anime	cartoon (100 %)	cartoon (100 %)
`pruebaAnime2.jpg`	anime	cartoon (100 %)	cartoon (100 %)
`pruebaAnime3.jpg`	anime	cartoon (100 %)	cartoon (98 %)
`pruebaAnime6.jpg`	anime	cartoon (100 %)	anime (99 %)
`pruebaPersona.jpg`	human	cartoon (100 %)	cartoon (97 %)
`pruebaPersona2.jpg`	human	cartoon (100 %)	human (82 %)

Este resultado es coherente con la naturaleza del problema. Dropout combate el sobreajuste, y en estos modelos la exactitud de entrenamiento y la de validación ya eran casi iguales, de modo que no había una brecha que cerrar. Reducir la coadaptación entre neuronas no corrige que las imágenes externas sean distintas a las del conjunto de datos, que es donde se concentra el error. Dropout se probó y se documenta, pero no aporta una mejora medible en este caso.

V-E. Tratamiento de los datos: estirar y aumento de datos

Los experimentos anteriores sugieren que el problema no está tanto en el modelo como en la poca diversidad de los datos de entrenamiento. Por ello se probaron dos cambios en el tratamiento de los datos, sobre EfficientNetB0 (el mejor modelo hasta ahora, sin Dropout, que no aportó mejora) y con el mismo entrenamiento en dos fases. Para poder explicar cuál de los dos cambios funciona, se probaron por separado y juntos, lo que da las cuatro variantes de la Tabla V.

TABLE V
VARIANTES DE TRATAMIENTO DE LOS DATOS (TODAS CON EFFICIENTNETB0)

Variante	Redimensionado	Aumento de datos
EfficientNetB0 (anterior)	padding	no
+ estirar	estirar	no
+ aumento (padding)	padding	sí
+ estirar y aumento	estirar	sí

Estirar. En lugar de conservar la proporción y rellenar con bandas, la imagen se redimensiona directamente a 128×128, con el mismo filtro de Lanczos. Esto elimina las bandas de relleno, que en el conjunto de datos solo aparecían en las imágenes de cartoon no cuadradas y podían servir de atajo, a cambio de deformar las imágenes que no son cuadradas.

Aumento de datos. Consiste en aplicar transformaciones aleatorias a las imágenes de entrenamiento, distintas en cada época, de modo que el modelo vea muchas variantes

de cada imagen. Se aplica imagen por imagen y solo en entrenamiento; la validación y la prueba no se modifican. Las transformaciones se eligieron para atacar los atajos identificados en el análisis exploratorio (composición y proporción, color y nitidez):

- volteo horizontal;
- rotación de hasta 11 grados;
- zoom de hasta 25 % en cada eje de forma independiente, lo que también varía la proporción de la imagen;
- traslación de hasta 10 %;
- cambios de brillo, contraste y saturación;
- conversión a escala de grises, con probabilidad de 0.15 (por ejemplo, para manga o fotografías antiguas);
- baja resolución, con probabilidad de 0.5: la imagen se reduce a entre 35 % y 100 % de su tamaño y se vuelve a ampliar, para que la nitidez no delate la clase.

Los huecos que dejan la rotación, el zoom y la traslación se rellenan reflejando la imagen. La Figura 10 muestra ejemplos.

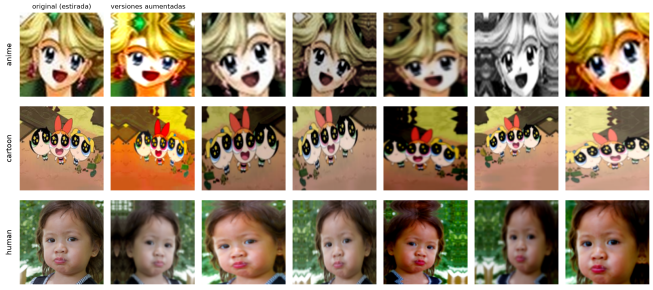


Fig. 10. Imagen original (estirada) de cada clase y seis versiones aumentadas.

La Figura 11 muestra el entrenamiento. Sin aumento de datos, la exactitud de entrenamiento llega a 100 % y la de validación queda por debajo. Con aumento, las curvas de entrenamiento y de validación casi se superponen y la exactitud de entrenamiento no llega a 100 % (99.87 % y 99.75 % al final), que es el efecto regularizador esperado.

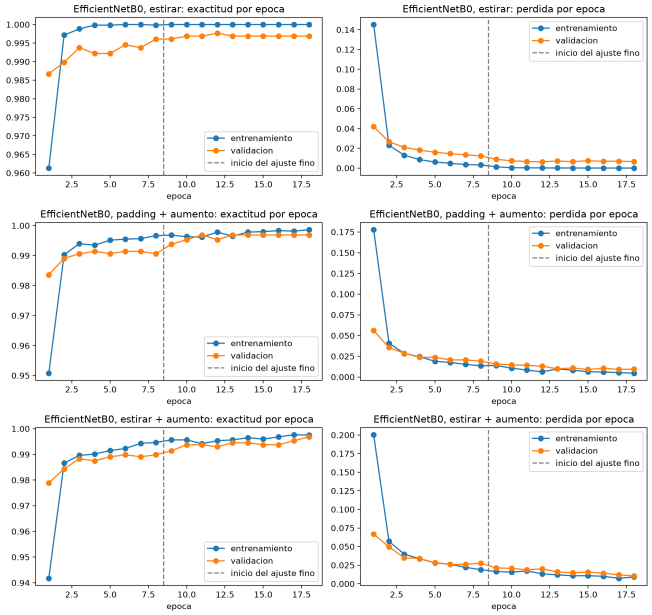


Fig. 11. Exactitud y pérdida por época de las tres variantes nuevas. La línea punteada marca el inicio del ajuste fino.

En validación y prueba, las tres variantes nuevas rinden ligeramente peor que el modelo original (Tabla VIII): 99.69 % frente a 99.92 % en validación, y entre 2 y 3 errores en prueba frente a 1. Es el costo esperable de entrenar con datos más variados, y son diferencias de una o dos imágenes.

Con las imágenes externas (Tabla VI), estirar por sí solo no cambia ningún acierto (4 de 8), pero reduce la seguridad de los errores: la ilustración anime `pruebaAnime2.jpg`, clasificada como cartoon con 100 % de confianza, baja a 84 %. El aumento de datos sí corrige una imagen, la viñeta de manga en blanco y negro (`pruebaAnime3.jpg`), con lo que se llega a 5 de 8 (exactitud de 62.5 % y F1 macro de 0.635), tanto con padding como estirando. Esto es coherente con que el aumento incluye imágenes en escala de grises, aunque este experimento no lo comprueba. Combinar ambos cambios no mejora el resultado respecto al aumento solo (5 de 8) y reduce todavía más la confianza de los errores (54 % en `pruebaAnime2.jpg`).

TABLE VI
PREDICCIONES DE LAS VARIANTES DE TRATAMIENTO DE DATOS SOBRE IMÁGENES EXTERNAS (ENTRE PARÉNTESIS, LA CONFIANZA)

Imagen	Real	Anterior	+ estirar	+ aumento	+ estirar y aumento
`Caricaturas2.jpg`	cartoon	cartoon (100 %)	cartoon (100 %)	cartoon (100 %)	cartoon (100 %)
`caricatura1.jpg`	cartoon	cartoon (100 %)	cartoon (100 %)	cartoon (100 %)	cartoon (99 %)
`pruebaAnime.jpg`	anime	cartoon (100 %)	cartoon (90 %)	cartoon (91 %)	cartoon (85 %)
`pruebaAnime2.jpg`	anime	cartoon (100 %)	cartoon (84 %)	cartoon (91 %)	cartoon (54 %)
`pruebaAnime3.jpg`	anime	cartoon (94 %)	cartoon (99 %)	anime (91 %)	anime (69 %)
`pruebaAnime6.jpg`	anime	anime (97 %)	anime (89 %)	anime (87 %)	anime (95 %)
`pruebaPersona.jpg`	human	cartoon (98 %)	cartoon (100 %)	cartoon (99 %)	cartoon (100 %)
`pruebaPersona2.jpg`	human	human (81 %)	human (100 %)	human (84 %)	human (96 %)

En todas las variantes siguen fallando las dos ilustraciones anime de estilo moderno y el modelo anatómico, y todos los errores se clasificaron como cartoon. La mejora es modesta y poco concluyente: es un acierto más en 8 imágenes, cada modelo se entrenó una sola vez y las diferencias en validación y prueba son de una o dos imágenes. Una

posible explicación de que el aumento de datos no corrija las ilustraciones anime modernas, que no se comprobó, es que en el conjunto de entrenamiento las imágenes de anime son solo rostros recortados: transformaciones como el zoom o la traslación varían lo que se ve de esos rostros, pero no crean personajes con cuerpo, fondos ni estilos que el conjunto no contiene. Resolverlo requeriría más diversidad en los propios datos de entrenamiento.

VI. Predicciones con imágenes nuevas

Las métricas de validación y de prueba se calculan con imágenes del mismo conjunto de datos, por lo que comparten con el entrenamiento los sesgos descritos en el análisis exploratorio (resolución, color y composición). Para comprobar si el modelo generaliza, se evaluó con imágenes que no pertenecen al conjunto de datos, con el mismo preprocesado que se usó al entrenar.

VI-A. Conjunto de imágenes externas

Se reunieron 10 imágenes, y la clase de cada una se deduce del prefijo de su nombre de archivo. Antes de evaluar, se compararon por contenido (resumen MD5) con las imágenes del conjunto de datos, y se encontró que dos de ellas son copias exactas de imágenes de la partición de entrenamiento: una de anime (`anime/1001_2000.jpg`) y una de human (`human/00000.png`). Como el modelo pudo verlas al entrenar, no sirven para medir la generalización y se excluyeron de la evaluación; se usaron solo como control, y el modelo las clasificó correctamente, como era de esperar. Quedan 8 imágenes externas: 4 de anime, 2 de cartoon y 2 de human. **[PENDIENTE: indicar el origen o la licencia de las imágenes externas.]**

Es un conjunto muy pequeño, por lo que permite observar patrones pero no estimar el desempeño con precisión. Además, una de las imágenes de human es un modelo anatómico de musculatura en 3D y no una fotografía, es decir, no corresponde al tipo de imagen de esa clase en el conjunto de entrenamiento.

VI-B. Resultados con imágenes externas

La Tabla VII muestra las predicciones de los dos modelos y la Figura 12 las de la CNN base. La CNN base acertó 2 de las 8 imágenes (exactitud de 25.0 % y F1 macro de 0.133), frente a 99.53 % de exactitud en el conjunto de prueba.

TABLE VII
Predicciones sobre imágenes externas (entre paréntesis, la confianza de EfficientNetB0)

Imagen	Descripción	Real	CNN base	EfficientNetB0
`Caricaturas2.jpg`	Collage de personajes de caricaturas	cartoon	cartoon	cartoon (100 %)
`caricatural.jpg`	Caricatura dibujada de una persona	cartoon	cartoon	cartoon (100 %)
`pruebaAnime.jpg`	Ilustración anime horizontal	anime	cartoon	cartoon (100 %)
`pruebaAnime2.jpg`	Ilustración anime de personaje	anime	cartoon	cartoon (100 %)
`pruebaAnime3.jpg`	Viñeta de manga en blanco y negro	anime	cartoon	cartoon (94.4 %)
`pruebaAnime6.jpg`	Rostro en arte pixelado	anime	cartoon	anime (97.1 %)
`pruebaPersona.jpg`	Modelo anatómico 3D	human	cartoon	cartoon (97.6 %)
`pruebaPersona2.jpg`	Fotografía de un hombre	human	cartoon	human (80.7 %)

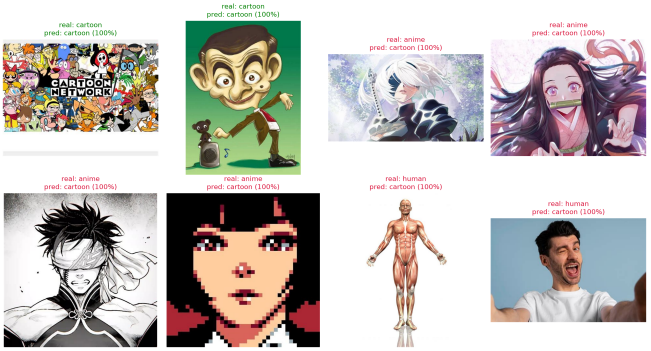


Fig. 12. Predicciones de la CNN base sobre las imágenes externas (en verde los aciertos y en rojo los errores).

En la CNN base, las dos imágenes acertadas son las de cartoon. Los 6 errores se clasificaron como cartoon, ninguno como anime ni como human, y todas las predicciones tienen una confianza de al menos 99.999 %, es decir, el modelo se equivoca con total seguridad.

Para saber si este resultado se debe al relleno (padding) de las imágenes no cuadradas, se repitió la evaluación con dos variantes sin barras de relleno: estirando la imagen a un cuadrado y recortando su centro. Las predicciones fueron las mismas en las tres variantes, por lo que el padding no explica estos errores.

El resultado indica que la exactitud casi perfecta en validación y prueba no se mantiene con imágenes de otro origen. Una posible explicación, que este experimento no comprueba, es que el modelo aprendió una noción estrecha de las clases anime y human (rostros recortados de un estilo y una toma particulares) y que cartoon, la clase más heterogénea según el análisis exploratorio, funciona como clase por defecto para todo lo que no se parece a esos ejemplos. Esto motivó probar una red preentrenada y las demás mejoras, que se comparan también con este conjunto externo y no solo con el de validación.

Con EfficientNetB0 se acertaron 4 de las 8 imágenes (exactitud de 50.0 % y F1 macro de 0.522). Además de las dos caricaturas, acertó el rostro en arte pixelado (anime) y la fotografía del hombre (human), esta última con la confianza más baja de todas (80.7 %). Siguió fallando con las dos ilustraciones anime de estilo moderno, la viñeta de manga y el modelo anatómico, y todos sus errores se clasificaron de nuevo como cartoon, aunque con una confianza algo menor en algunos casos (94.4 % y 97.6 %). Con solo 8 imágenes, la diferencia de 2 aciertos no es concluyente: sugiere que la transferencia de aprendizaje ayuda algo, pero no resuelve el problema. Una posible explicación, que no se comprobó, es que el clasificador se entrena con ejemplos de anime y human muy estrechos (rostros recortados de un estilo particular), por lo que las características preentrenadas por sí solas no corrigen la falta de diversidad de los datos.

VI-C. Interfaz web

Las predicciones también pueden realizarse mediante una interfaz web. Un servidor en Flask expone los modelos entrenados, y una página (HTML, CSS y JavaScript) permite

elegir el modelo en un selector, subir una imagen y ver la clase predicha, la probabilidad de cada clase y la imagen tal como la ve el modelo. Al cambiar de modelo con una imagen ya cargada, la predicción se recalcula. El servidor usa el mismo módulo de preprocesado que el entrenamiento, y a cada modelo le aplica el redimensionado con que se entrenó (padding o estirar), para que reciba lo mismo que vio al aprender.

VII. Resultados

La Tabla VIII resume la exactitud y el F1 macro de los modelos en los tres conjuntos de evaluación, y la Figura 13 muestra sus matrices de confusión en el conjunto de prueba.

TABLE VIII

Exactitud y F1 macro por modelo y conjunto de evaluación

	Validación		**Prueba**		**Externas (8)**	
Modelo	Exactitud	F1	Exactitud	F1	Exactitud	F1
CNN base (desde cero)	0.9984	0.9985	0.9953	0.9953	0.2500	0.1333
CNN + Dropout	0.9984	0.9985	0.9961	0.9960	0.2500	0.1333
EfficientNetB0 (transfer learning)	0.9992	0.9992	0.9992	0.9992	0.5000	0.5222
EfficientNetB0 + Dropout	0.9992	0.9992	1.0000	1.0000	0.5000	0.5222
EfficientNetB0 + estirar	0.9969	0.9968	0.9984	0.9984	0.5000	0.5222
EfficientNetB0 + aumento (padding)	0.9969	0.9969	0.9984	0.9984	0.6250	0.6349
EfficientNetB0 + estirar y aumento	0.9969	0.9968	0.9977	0.9976	0.6250	0.6349

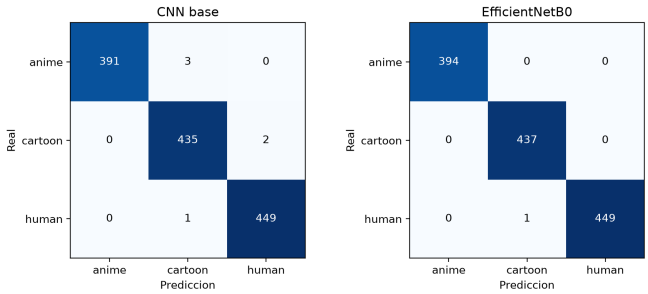


Fig. 13. Matrices de confusión de la CNN base y de EfficientNetB0 en el conjunto de prueba.

En validación y en prueba todos los modelos superan el 99.5 % y las diferencias entre ellos son de una a tres imágenes, por lo que esos conjuntos no permiten distinguir entre modelos. Con las imágenes externas, en cambio, el desempeño cae de forma notable y sí aparecen diferencias: 25.0 % en la CNN (con o sin Dropout), 50.0 % en EfficientNetB0 (con o sin Dropout, o estirando) y 62.5 % con aumento de datos (con padding o estirando). Dropout no cambió ninguna predicción externa. La transferencia de aprendizaje y el aumento de datos mejoraron el resultado externo en 2 y en 1 de las 8 imágenes, respectivamente, lo cual no es concluyente con tan pocas imágenes. Ningún modelo resolvió las ilustraciones anime de estilo moderno. Por eso, la evaluación con imágenes externas es la que se usa para comparar los modelos.

[PENDIENTE: agregar las conclusiones finales.]

References

[1] hadiepratamatulili, "Anime vs Cartoon vs Human," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/hadiepratamatulili/anime-vs-cartoon-vs-human [Accessed: Oct. 26, 2026].

[2] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org/

[3] F. Chollet et al., "Keras," 2015. [Online]. Available: https://keras.io

[4] Pillow (PIL Fork), "2.7.0 release notes," Pillow Documentation. [Online]. Available: https://pillow.readthedocs.io/en/stable/releasenotes/2.7.0.html

[5] G. Parmar, R. Zhang and J.-Y. Zhu, "On aliased resizing and surprising subtleties in GAN evaluation," arXiv:2104.11222, 2021. [Online]. Available: https://arxiv.org/abs/2104.11222

[6] G. Gando, T. Yamada, H. Sato, S. Oyama and M. Kurihara, "Fine-tuning deep convolutional neural networks for distinguishing illustrations from photographs," *Expert Systems with Applications*, vol. 66, pp. 295–301, 2016, doi: 10.1016/j.eswa.2016.08.057.

[7] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, 2019, pp. 6105–6114.

[8] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.